

Negative-Weight Reweighting for V+jets

Diagnosis, training, validation and the limit — the complete fix

Chirayu Gupta — VUB

2026-07-31

Method: Palmer & Kronheim, [arXiv:2510.16217](https://arxiv.org/abs/2510.16217)

H+c → WW, 2022postEE

The headline

builder	baseline	negrw	Δ
v11	1742	1343	-23%
v32	1935	1491	-23%

Expected 95% CL upper limit r_{95} , blind Asimov.

The $H+c \rightarrow WW$ limit was **not** limited by physics systematics — it was limited by **Monte-Carlo statistical noise** in the V+jets template, caused by amc@NLO negative weights.

Learning *where the negative weights live* and replacing the random $\pm$ cancellation with its expectation removes the noise **at the source**, without touching the physics.

Same yield. Same shape. Much smaller variance.

Part 1 — The problem

The estimator is a cancellation

amc@NLO events carry **signed** weights. The standard yield estimator

$$\hat{N}_B = \sum_{i \in B} w_i, \quad w_i = \pm |w_i|$$

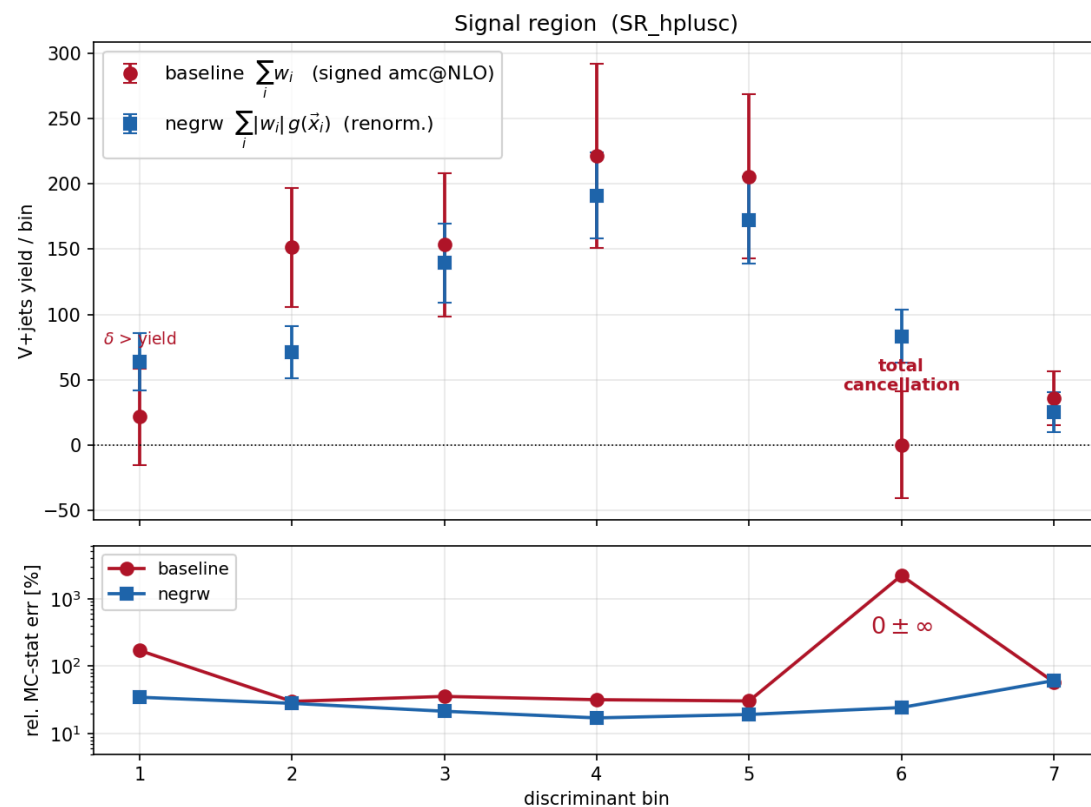
is a *difference of two large numbers*.

- The **expectation** is correct — the yield is unbiased.
- The **variance** is not: it scales with $\sum w_i^2 = \sum |w_i|^2$, i.e. with the *unsigned* statistics, while the mean is the much smaller signed sum.
- Effective statistics collapse: $N_{\text{eff}} = (\sum w)^2 / \sum w^2 \ll N$.

16.4% of our V+jets events carry a negative weight.

In a sparse bin, the positives and negatives can cancel **completely** — leaving a bin with zero content and a finite, large error.

What that does to the actual template



V+jets in the SR, **from the real combine input file**. Bin 6 is a

total cancellation — content floored to 10^{-6} with error ± 41 ,

i.e. a relative MC-stat error of 4×10^9 %. Bin 1 has error larger than its yield.

The consequence in the fit

`autoMCStats` gives every template bin a nuisance sized by its MC-stat error. A bin with $0 \pm \infty$ hands the fit an **unconstrained** background parameter sitting directly under the signal.

	v11	v32
stat-only floor	771	600
freeze <code>autoMCStats</code>	1032	1068
full	1742	1935
autoMCStats inflation	+710	+867

~81% of the total uncertainty traced to V+jets undersampling in the SR.

The gap between "freeze autoMCStats" and "full" **is** the problem — it is bigger than every physics systematic combined.

Part 2 — The method

Replace the random sign with its expectation

Train a classifier for the probability that an event has a positive weight, given **generator-level** kinematics $\vec{x}$ only:

$$P_+(\vec{x}) = P(\text{genWeight} > 0 \mid \vec{x})$$

Define the reweight factor and the new estimator

$$g(\vec{x}) = 2P_+(\vec{x}) - 1 = P_+ - P_-, \quad \hat{N}_B = \sum_{i \in B} |w_i| g(\vec{x}_i)$$

Why the mean is unchanged: $g(\vec{x})$ is exactly the *expected sign* of an event at $\vec{x}$. Summing $|w| \cdot \langle \text{sign} \rangle$ instead of the realised $\pm |w|$ replaces a random quantity by its mean.

Why the variance drops: every event now contributes with the **same sign**, scaled by a smooth factor. No cancellation is left to fluctuate. The gain is largest exactly where the cancellation was worst — sparse, hard bins.

It is an algebraic identity, not an approximation

From the paper (their Eqs. 2–6). Write the (signed) generator density as a difference of two normalised positive densities:

$$\text{PDF}(\vec{x}) = a \text{PDF}_+(\vec{x}) - b \text{PDF}_-(\vec{x}), \quad a, b \geq 0, \quad a - b = 1$$

The positive-weight probability at $\vec{x}$ is then

$$P_+(\vec{x}) = \frac{a \text{PDF}_+}{a \text{PDF}_+ + b \text{PDF}_-} \implies \text{PDF}(\vec{x}) = \underbrace{(2P_+(\vec{x}) - 1)}_{g(\vec{x})} \cdot (a \text{PDF}_+ + b \text{PDF}_-)$$

The right-hand factor is the **unsigned** density — which is what $\sum |w|$ samples.

So $\sum |w| g$ and $\sum w$ estimate *the same distribution*.

Caveat that drives everything downstream: this is exact for the **true** P_+ . We use an estimate $\hat{P}_+$ from a finite classifier, so closure becomes an **empirical** property that has to be measured — hence the validation in Part 4.

Part 3 — Training

Training region: train loose, infer tight

$g(\vec{x})$ is a **generator property** — it does not know about the analysis selection.

So we can train on a far larger sample than the SR.

	Training region	Inference region
Selection	base cuts + veto_emu_sr	tight eμ signal region
Meaning	veto <i>only</i> the exactly-one-eμ-pair topology	the analysis SR
Statistics	9,832,308 events (502 parquet)	~10 ⁴ events

- **Event-disjoint by construction** (paper §V.4) → no train/infer overfitting bias, no hold-out needed.
- Disjoint *events*, **not** orthogonal *phase space* — same generator physics, so g transfers.
- **Veto integrity verified:** 0 eμ SR events leaked into training.
- **Coverage verified:** SR-proxy $\vec{x}$ lies inside the training domain in every gen feature → **interpolation, never extrapolation.**

Inputs: 20 generator-level features

LHE event-level (10)

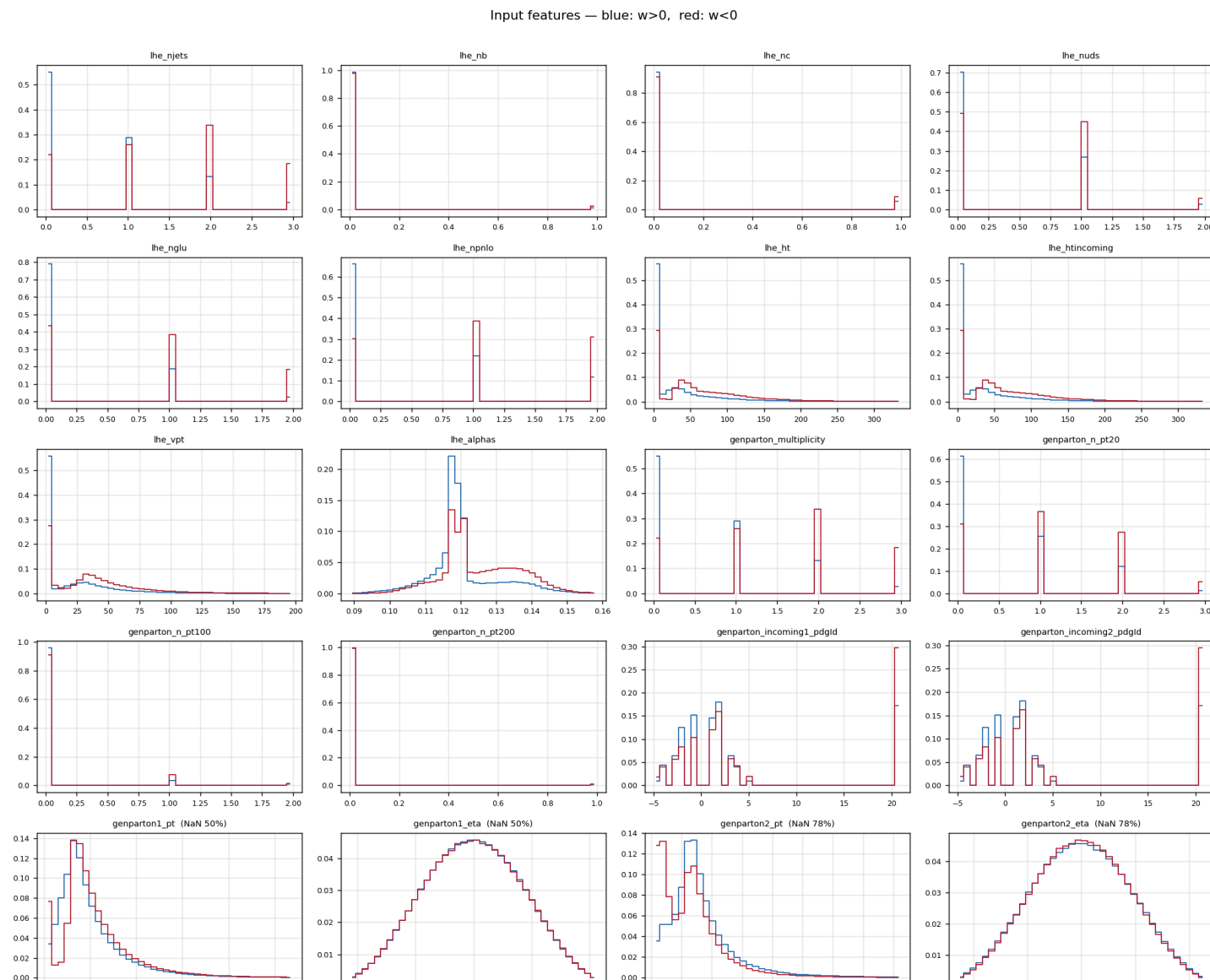
lhe_njets, lhe_nb, lhe_nc, lhe_nuds,
lhe_nglu, lhe_npnlo, lhe_ht,
lhe_htincoming, lhe_vpt, lhe_alphas

Hard-process gen partons (10)

genparton_multiplicity,
genparton_n_pt20 / n_pt100 / n_pt200,
genparton_incoming1_pdgId, genparton_incoming2_pdgId,
genparton1_pt, genparton1_eta,
genparton2_pt, genparton2_eta

- The paper's hand-built **parton-count / HT / V-pT / merging** variables.
- **Label:** `weight_nominal > 0` — the fully-populated signed amc@NLO weight.
Every event is usable; nothing is thrown away.
- Events with < 2 hard partons have **missing** parton kinematics → left as `NaN`,
handled **natively** by the histogram GBDT (no imputation, no sentinel value).
- **Only generator quantities.** Nothing reconstructed enters g ,
so the reweighting cannot sculpt the analysis observable.

The two weight classes really are separated



The model

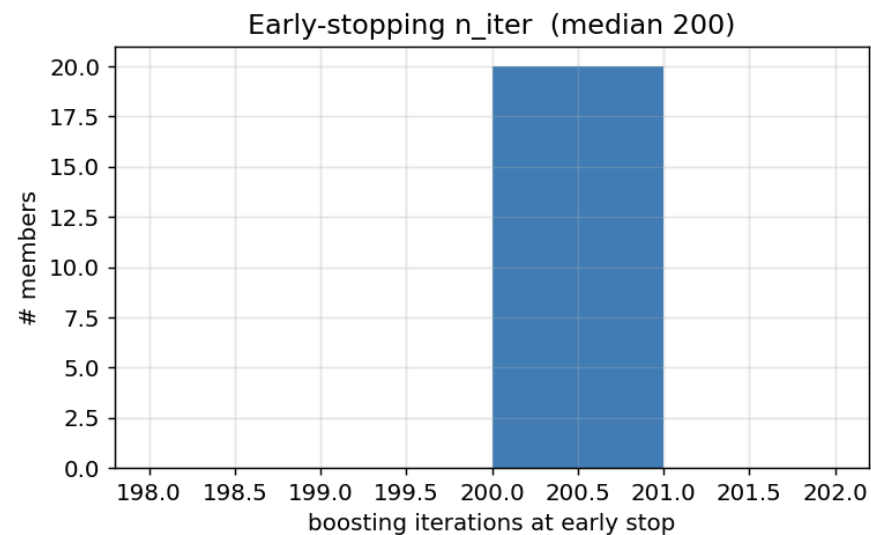
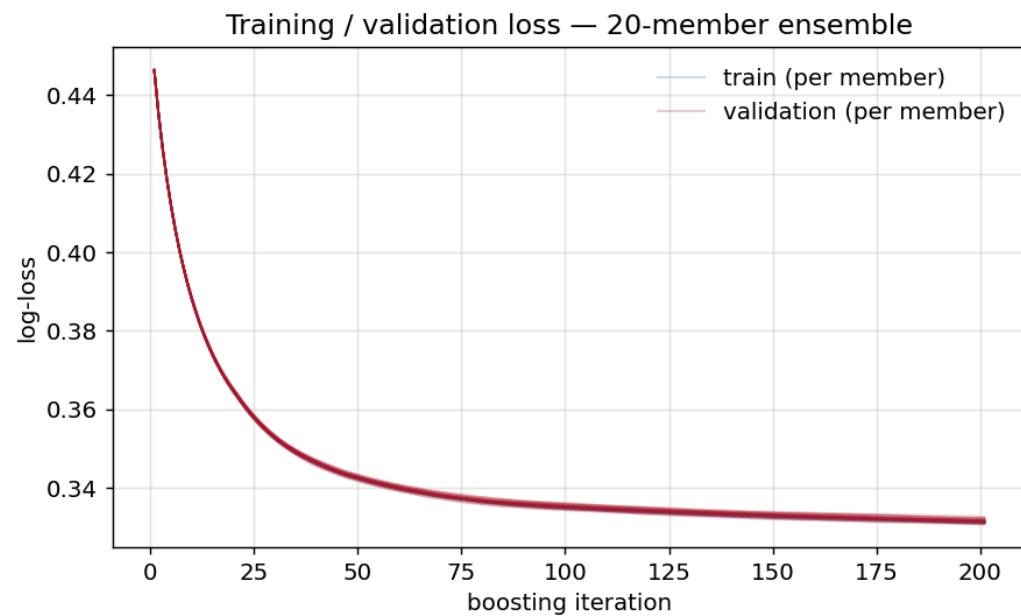
HistGradientBoostingClassifier (scikit-learn) — histogram-binned GBDT: bins each feature into ~256 buckets, grows shallow trees. Scales to ~10M events, handles `NaN` natively.

parameter	value
<code>loss</code>	<code>log_loss</code>
<code>max_iter</code>	200
<code>learning_rate</code>	0.05
<code>max_depth</code>	4
<code>l2_regularization</code>	1.0
<code>early_stopping</code>	True (<code>val_frac</code> 0.15)

Ensemble of 20 classifiers

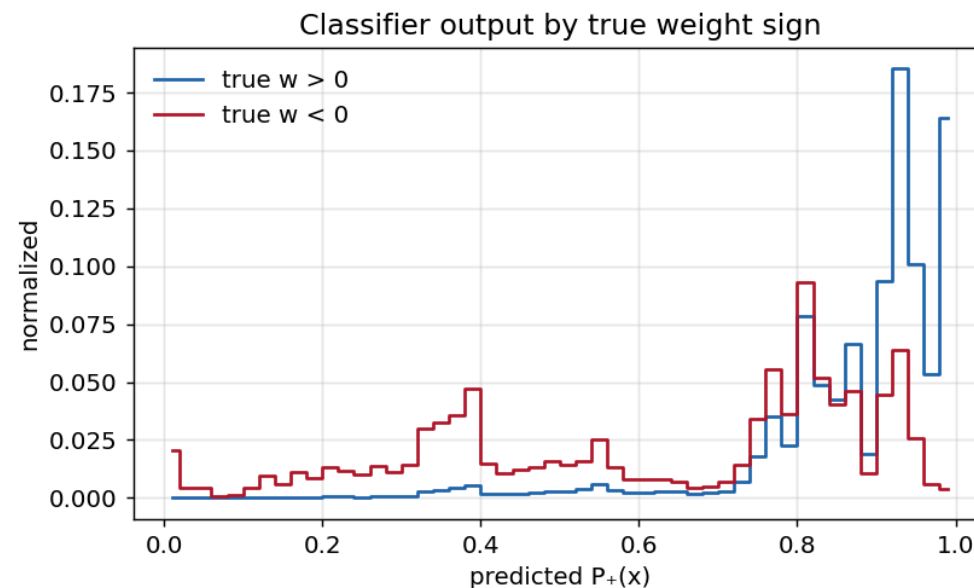
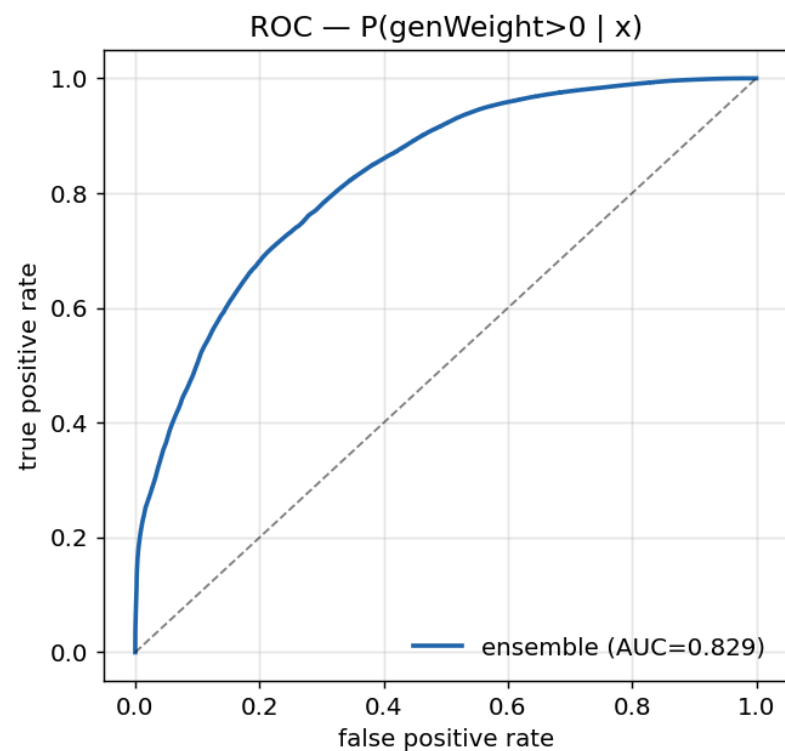
- Each on an independent **60% subsample**, drawn without replacement, different seed.
- `log_loss` $\Rightarrow$ the output is a **calibrated probability** — mandatory, since P_+ is used as a probability, not as a ranking score.
- **Why 20?** The spread across members, $\delta g = 2 \text{std}(P_+)$, is the method's own uncertainty. The ensemble doesn't just give a better central value — it **generates its own error band**.

Training convergence



- Smooth convergence; **train and validation curves lie on top of each other** → no overfitting (shallow depth-4 trees + L2 = 1.0 are doing their job).
- Loss still gently descending at iteration 200 → **early stopping never fires; all 20 members hit the `max_iter` ceiling** (median `n_iter` = 200).
- **Known headroom:** more iterations / deeper trees could still improve P_+ . Not pursued — the limit is already dominated by other things (Part 5).

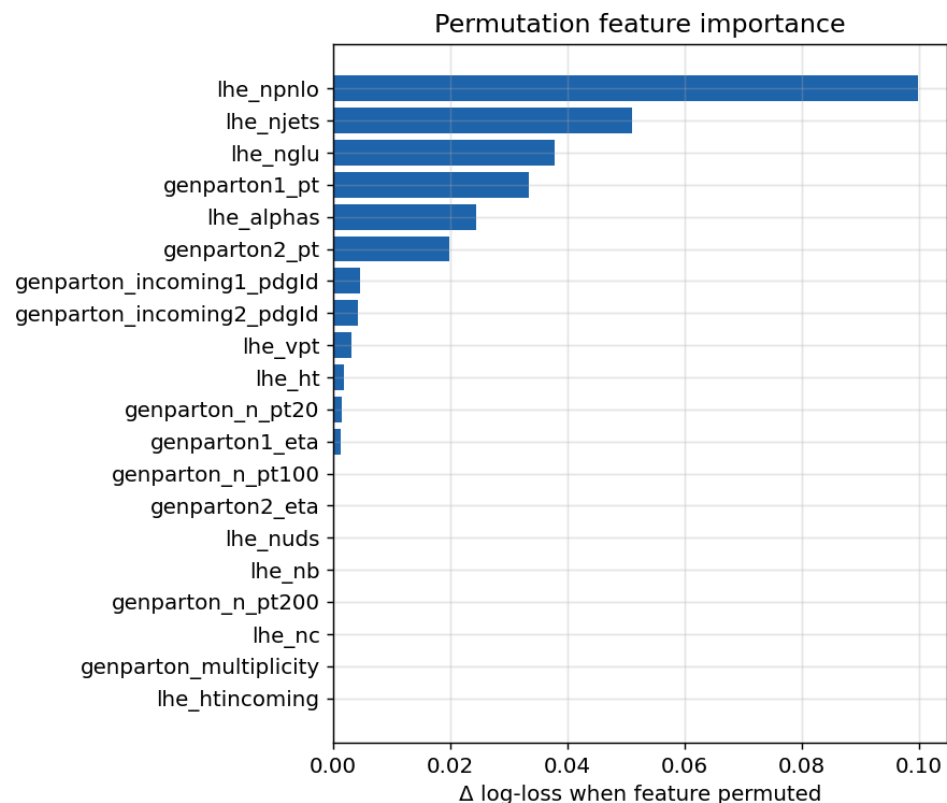
Classifier performance



metric	value
ensemble log-loss	0.331
ensemble AUC	0.829

True $w > 0$ events pile up at $P_+ \rightarrow 1$; $w < 0$ events spread to low P_+ .

What drives the prediction?



Permutation importance = increase in log-loss when a feature is scrambled.

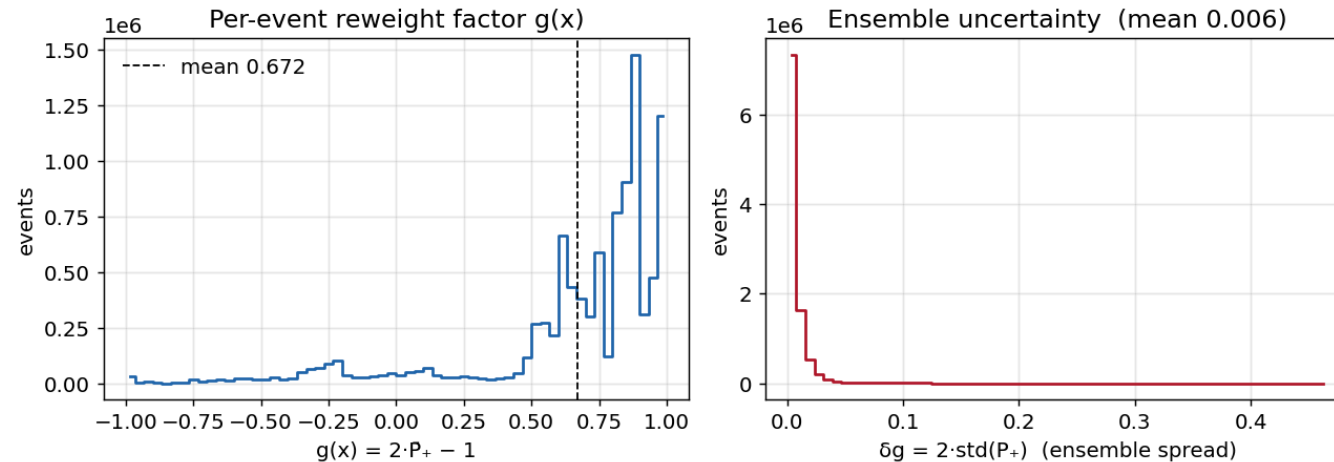
Physically sensible ordering:

1. `lhe_npnlo` — the **NLO merging variable**, dominates by far
2. `lhe_njets` — parton multiplicity
3. `lhe_nglu` — gluon count
4. `genparton1_pt` — leading parton hardness
5. `lhe_alphas` , `genparton2_pt`

These are exactly the variables that control where amc@NLO **negative weights** are generated — the merging and subtraction regions.

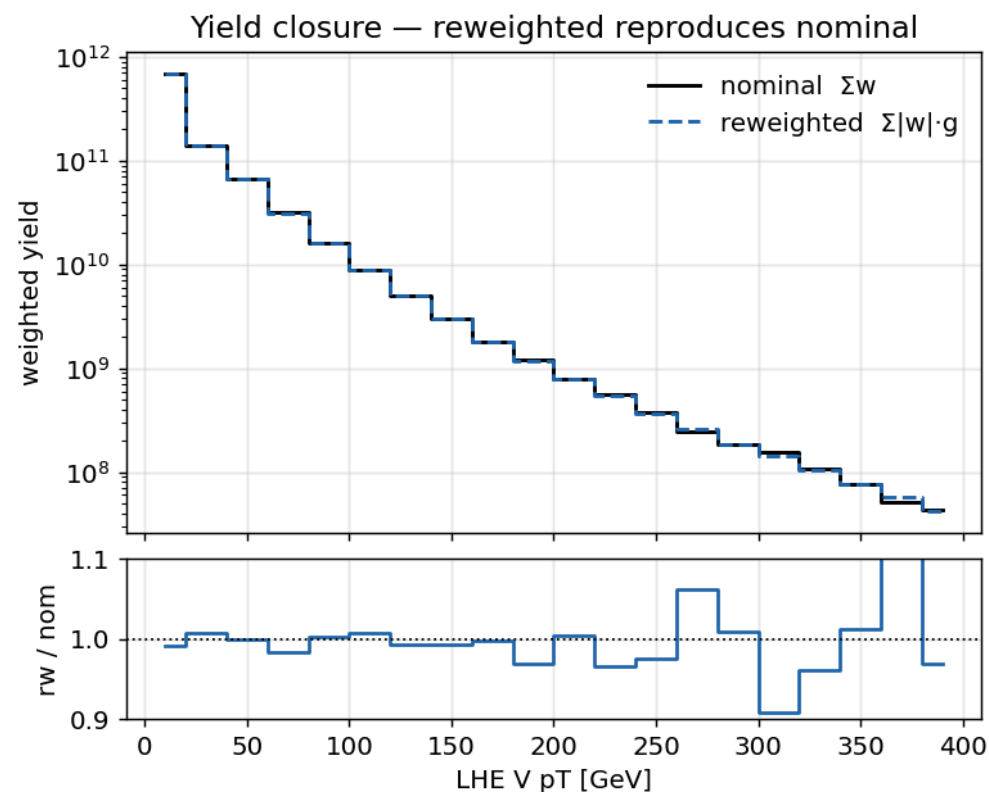
→ **The model learned real generator structure, not noise.**

The reweight factor g and its uncertainty



	value	reading
g mean	0.672	$= 2 \times 0.836 - 1$ — matches the global positive fraction ✓
g range	$[-0.991, 0.993]$	well inside the physical $[-1, 1]$; no pathological events
δg mean / max	0.006 / 0.467	20-model agreement is tight

Closure on the training region



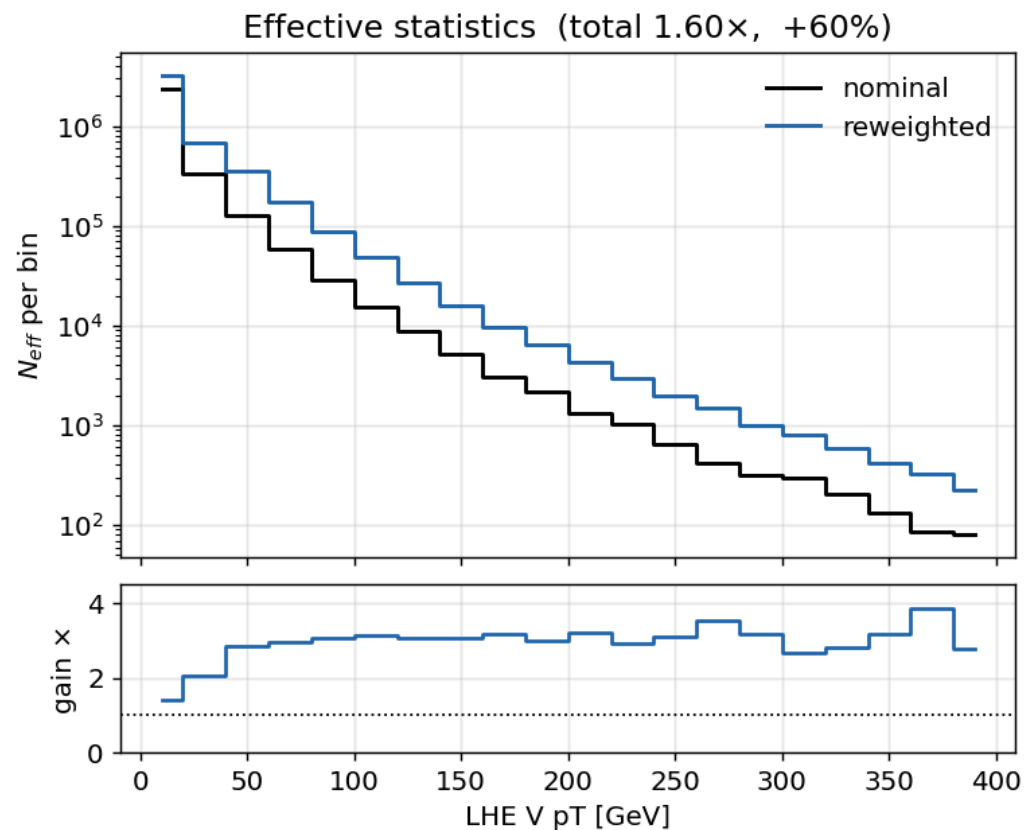
$$\frac{\sum |w| g}{\sum w} = \mathbf{0.994}$$

Ratio flat at unity across the whole V-pT spectrum, including the hard tail.

The reweighting **preserves the physics** — it removes variance and nothing else.

This is the primary gate on the method: if $\hat{P}_+$ were mis-modelled, this ratio would drift away from 1.

The payoff: effective statistics



Total: N_{eff} 2.92M $\rightarrow$ 4.68M (+60%)

~3× gain sustained across the entire hard V-pT tail
(2.8–3.8× for $p_T^V > 40$ GeV).

The bulk (first bin) gains least — it was never starved. The gain grows **exactly where autoMCStats hurts.**

That is the whole design: this method does nothing where you don't need it.

Per-bin gain in the tail

p_T^V [GeV]	N_{eff} nominal	N_{eff} reweighted	gain
0–20	2,341,819	3,262,187	1.39×
40–60	125,911	357,341	2.84×
80–100	28,976	88,889	3.07×
120–140	8,759	26,802	3.06×
160–180	3,068	9,671	3.15×
200–220	1,331	4,271	3.21×
260–280	420	1,488	3.54×
340–360	134	424	3.18×
360–380	85	326	3.84×

In the **actual SR** the gain is even larger: **3.44×**

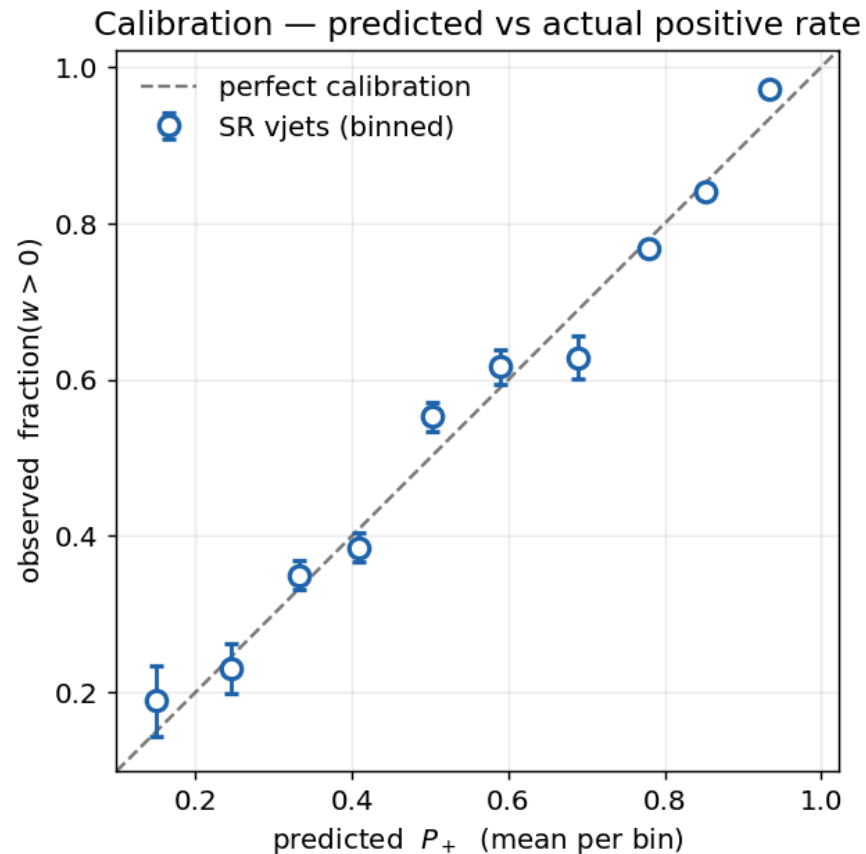
(455 → 1563) — the SR is precisely the starved regime the method targets.

Part 4 — Validation on the real SR

The model was trained on the veto- $e\mu$ region.

Everything here is measured on the **tight $e\mu$ SR it never saw** (10,205 V+jets events).

Is P_+ calibrated out-of-training-region?



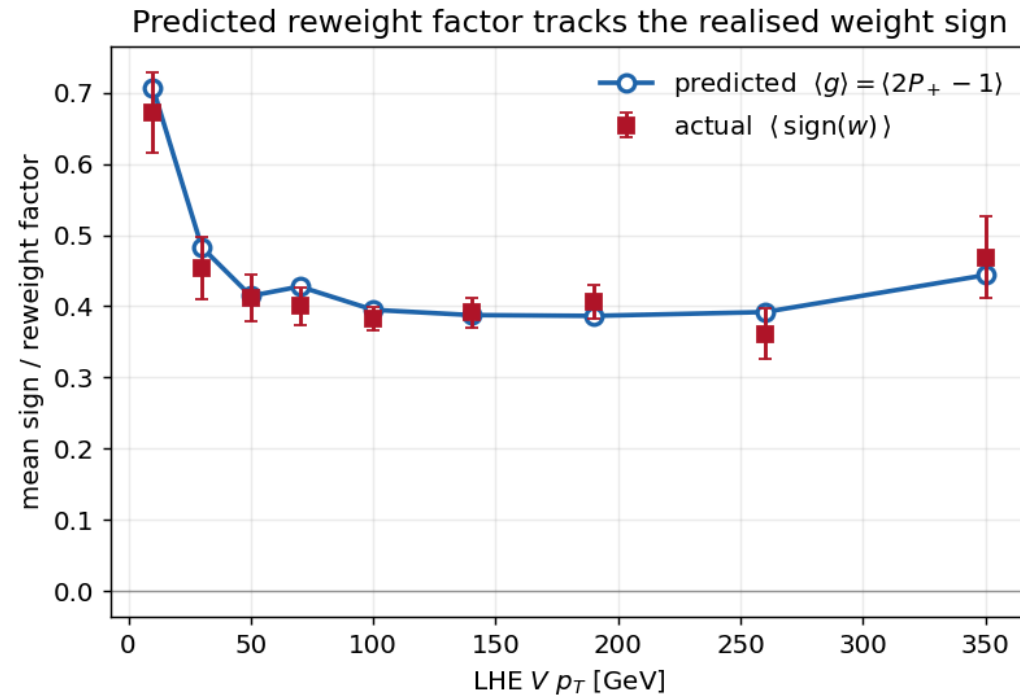
Bin SR events by **predicted** P_+ ; plot the **observed** fraction with $w > 0$ in each bin.

Points land on the diagonal across the full range ($P_+ \approx 0.15 \rightarrow 0.93$).

The predicted positive-rate is the realised one — in a region the classifier was never trained on.

This is the check that matters most: $g = 2P_+ - 1$ is only meaningful if P_+ is a probability, not merely a good ranking.

Does g track the realised sign?

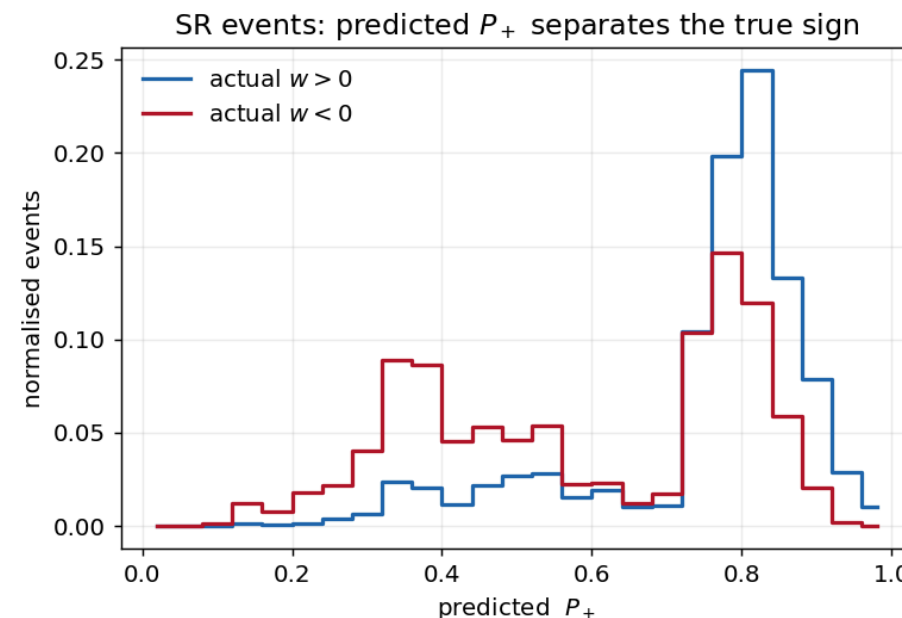
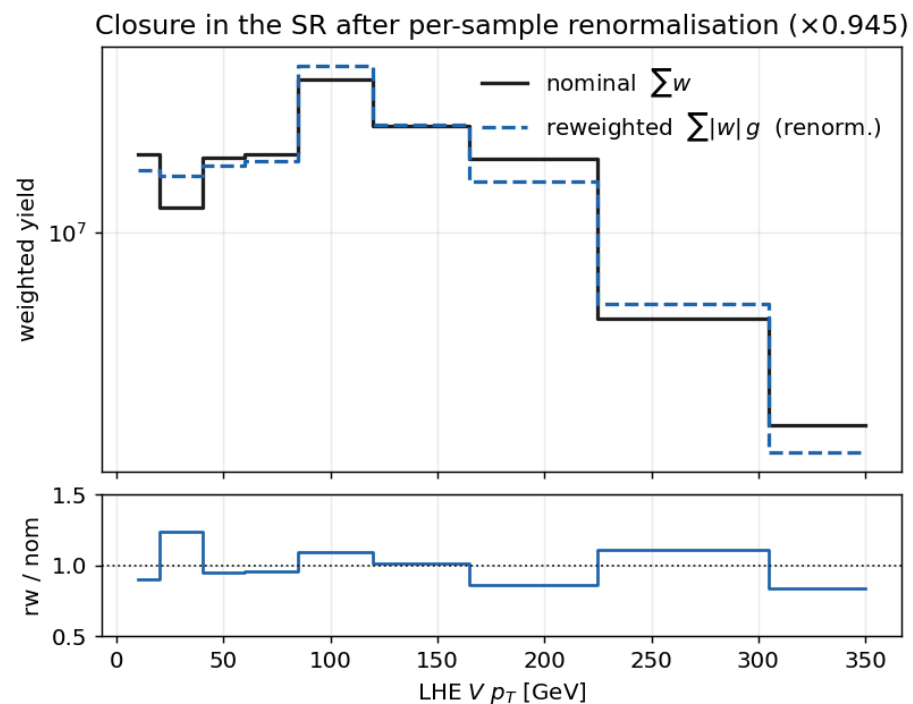


Predicted $\langle g \rangle = \langle 2P_+ - 1 \rangle$ (blue) vs the **actual** $\langle \text{sign}(w) \rangle$ (red), in bins of p_T^V .

Every measured point sits on the predicted curve — at **every** p_T^V , including the hard tail where the statistics are thinnest and the reweighting matters most.

The reweight factor reproduces the realised weight sign differentially, not just on average.

SR closure and sign separation



True $w > 0$ (blue) peak at high P_+ ; true $w < 0$ (red) shift low — on SR events.

After per-sample renorm: reweighted $\sum |w|g$ tracks nominal $\sum w$, ratio ≈ 1 .

The SR closes to **~6%** before renormalisation

(training-region closure was 0.994 on 9.8M events). The SR is a small, kinematically-biased corner of the training domain, so per-event g does not perfectly average to the *local* positive fraction. Next slide: what we do about it.

The one honest caveat: the SR closure offset

dataset	rows	$\sum w g / \sum w$
DYto2L_2Jets_50	9,646	1.014
WtoLNu_2Jets	485	1.133
DYto2L_2Jets_10to50	74	2.24 (74 rows — noise)
total	10,205	1.058

Decision: renormalise the reweighted V+jets template to the nominal yield, per dataset.

Applied factors: DYto2L_50 $\times 0.986$, WtoLNu $\times 0.900$, DYto2L_10to50 $\times 0.446$.

Why it is defensible

- Reweighting is meant to fix **variance, not yield** — exact closure is what the identity guarantees in the large- N limit. The rescale forces our finite- N estimate back onto it.
- Standard shape-only template practice (the existing DY smoothing does the same).
- It **keeps the entire benefit**: N_{eff} comes from reduced per-bin variance, which a global rescale does not touch.

Why it must be stated

- **This is our extension, not the paper's.** Their closure was clean enough that they apply no post-hoc rescale.
- The WtoLNu 13% is the one number worth revisiting (feature coverage of the W corner of the SR).
- It should be declared as such in the AN / thesis.

The method's own uncertainty — and why it's negligible

`CMS_negrw_vjets` : a **V+jets-only shape nuisance** built from the 20-model ensemble spread,

$$\text{Up/Down} = |w_{\text{nom}}| \cdot \text{clip}(g \pm \delta g, -1, 1),$$

each renormalised to the nominal yield $\Rightarrow$ **shape-only** (paper §IV C, event-level).

	v11	v32
limit without the nuisance	1343	1491
limit with it	1343	1491

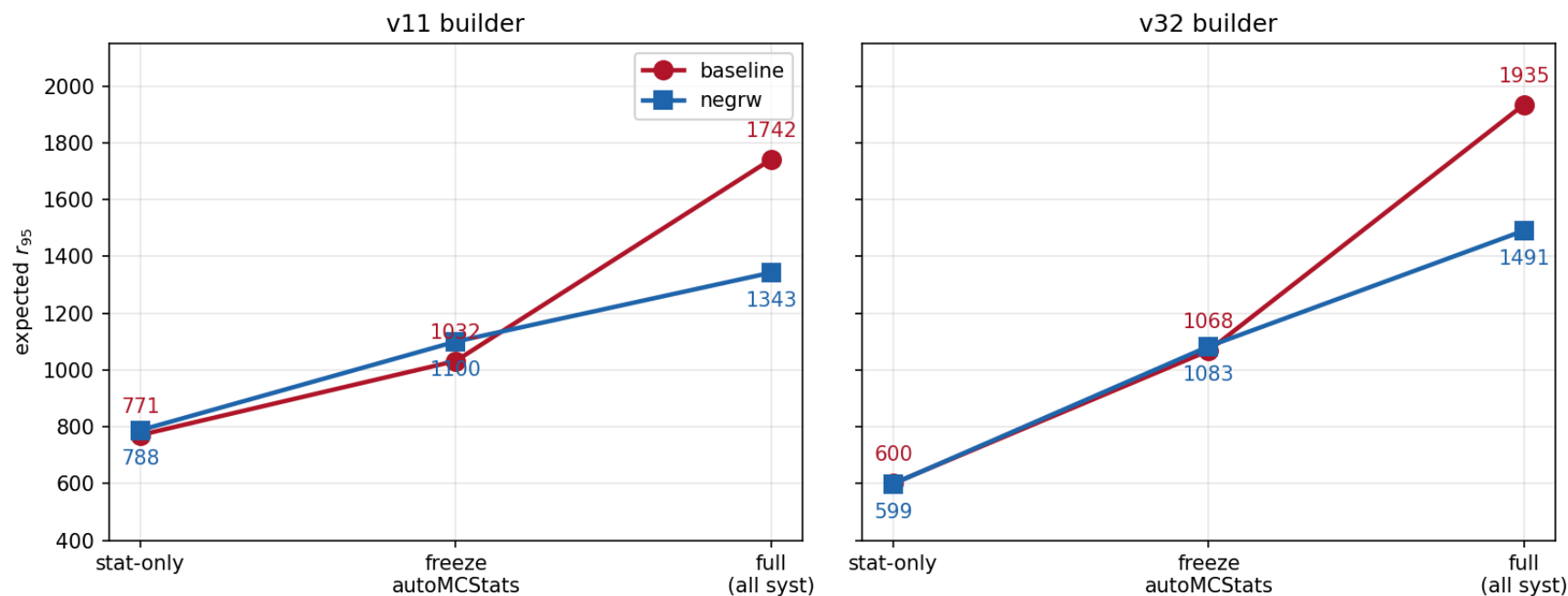
Max per-bin effect $\pm 1.3\%$ (SR V+jets Up 730.5 / nom 735.4 / Down 740.5),

against per-bin MC-stat still $\sim 20\%$. δg mean ≈ 0.012 — the 20 members agree tightly — so combine simply profiles it away. **1343 / 1491 are therefore honest numbers that already include the method's own uncertainty.**

Part 5 — Result

Where the improvement comes from

The autoMCStats inflation is what collapses



The two curves are **on top of each other** at stat-only and at freeze-autoMCStats, and separate only at "full". The entire gain is the collapse of the autoMCStats inflation — nothing else moved.

The numbers

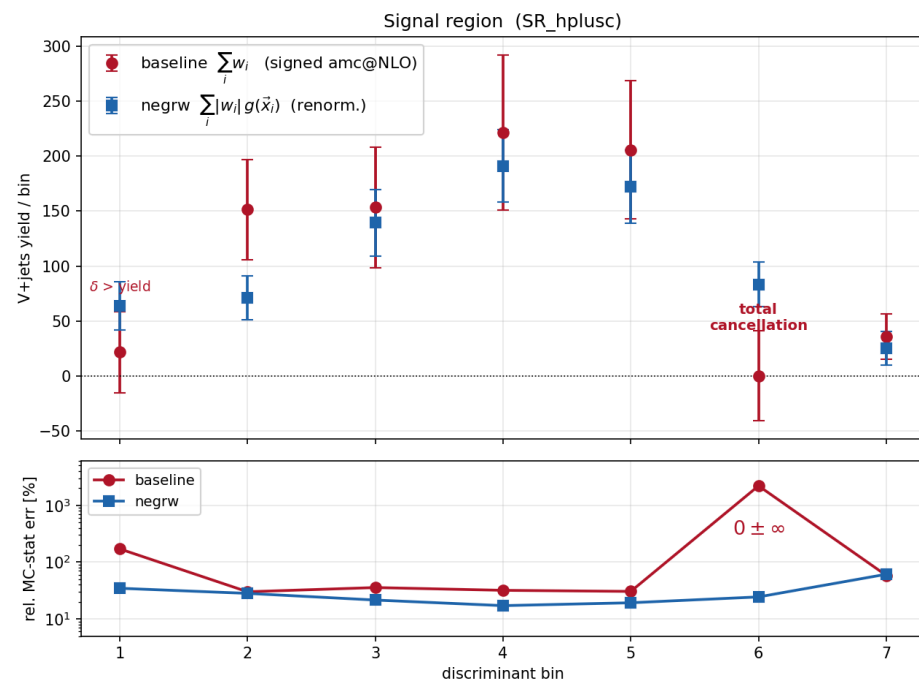
builder	metric	baseline	negrw	Δ
v11	full	1742	1343	-23%
	stat-only	771	788	+2%
	freeze autoMCStats	1032	1100	+7%
	autoMCStats inflation	710	243	-66%
v32	full	1935	1491	-23%
	stat-only	600	599	flat
	freeze autoMCStats	1068	1083	+1%
	autoMCStats inflation	867	408	-53%

Why stat-only moves +2% for v11 and not for v32: the "stat-only" limit

freezes `allConstrainedNuisances` but **keeps** `prop_bin*` floating — so it still contains per-bin MC-stat error. Changing those errors (bin 6: $0 \pm \infty \rightarrow 91 \pm 25\%$) *should* move it.

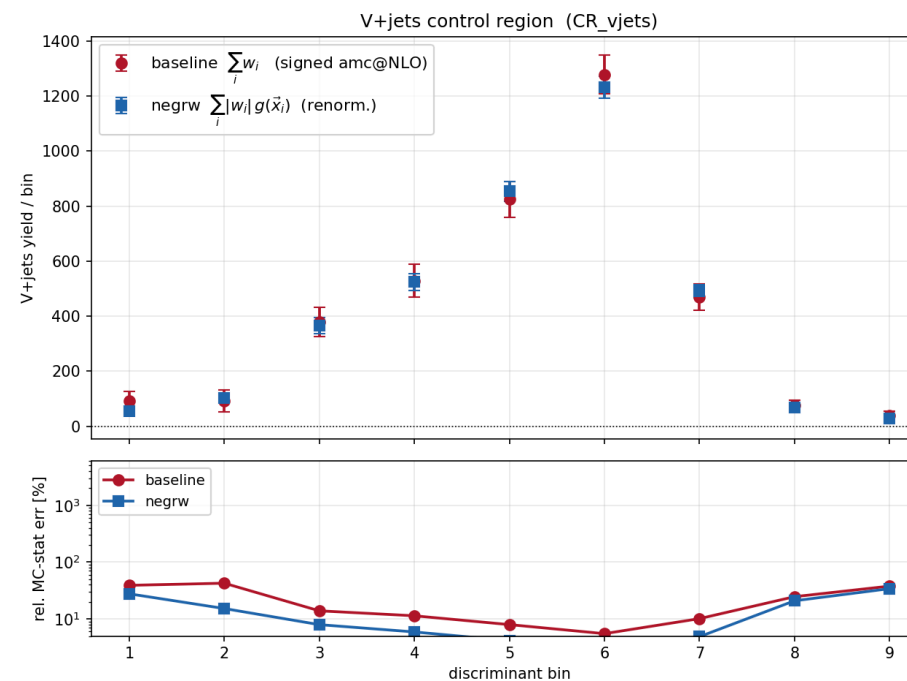
It is a rebalance, not a distortion.

The template-level evidence



SR: mean rel. MC-stat error over usable bins **59.6% → 30.3%**, plus one bin recovered from $0 \pm \infty$.

This is the mechanism, measured directly on the combine inputs — not inferred from the limit. The limit improvement is a *consequence* of these two plots.



CR_vjets: **21.4% → 13.8%**. Every bin's error shrinks; contents stay physical; yield preserved by the renorm.

Reproducibility — the version trap

The ensemble is trained **inside the same Singularity image the analysis workers run** (`coffea-base-almalinux9:0.7.30-py3.10` , **scikit-learn 1.7.2**), submitted as a Condor job.

- A model pickled by a *different* sklearn version **fails to load on the workers** (`AttributeError: __pyx_unpickle_CyHalfBinomialLoss`). Version match is **mandatory, not cosmetic** — and it fails at run time on the grid, not at build time.
- Corollary: the local `b_hive` env (sklearn 1.4.2) can no longer load this model. Local processor tests must run **inside the image**.
- The model must live on **AFS**, not EOS — the worker container cannot read arbitrary EOS paths (`PermissionError`). Master copy stays on EOS; workers read the AFS stage.
- Reproduces the earlier reference run to 3 decimals (closure 0.994, N_{eff} +60%).

Three real bugs found before the physics was trusted

1. The dataset gate was substring-matching — and would have reweighted SIGNAL.

`any(p in dataset for p in ["DYto2L", "WtoLNU"])` matches

`WplusH_WtoLNU_Hto2Wto2L2Nu` → the **WH signal** template would have been silently reweighted with a V+jets generator model. Fixed to exact matching against the three exact dataset names.

2. The exact match was then too strict and silently did nothing.

Condor passes `--dataset DYto2L_2Jets_50_7` (partition suffix), so exact matching rejected *everything* — the first submit wrote parquets with **no** `weight_negrw` **at all** and no error.

Fixed by stripping a trailing `_\d+` before matching, still anchored so WH stays untouched.

3. Model on EOS → `PermissionError` inside the container. Every Condor job would have died.

Lesson adopted: Condor jobs **overwrite** their output dir on write, so a buggy submit is itself destructive. Run **one canary job and verify the columns** before any full submit over live data.

Implementation

The trained ensemble is applied **inside the coffea processor**, adding **2 columns** to the V+jets SR parquets:

column	meaning
<code>weight_negrw</code>	$g = 2\bar{P}_+ - 1$ — the per-event reweight factor
<code>weight_negrw_std</code>	$\delta g = 2 \text{std}(P_+)$ — ensemble spread → shape systematic

- **Config-gated + dataset-gated:** the `negrw:` block in `hww_combine_fixed.yaml` lists three exact dataset names → **V+jets only**; nothing else in the analysis is touched.
- Combine fills the V+jets template with $|w_{\text{nom}}| \cdot g \cdot (\text{renorm})$ and adds the `CMS_negrw_vjets` shape nuisance from $\pm \delta g$.
- Object-shift call sites do **not** get negrw Up/Down — no double counting.
- **Reweighting REPLACES smoothing.** Both regularise the same sparse tail; stacking them double-counts and muddies interpretation. `dy_template_smooth.py` is not run.

Status

Done

- Training region designed + validated (0 ep leaked; interpolation-only coverage)
- 20-model ensemble trained, version-matched to the workers
- Closure 0.994; N_{eff} +60% ($\sim 3\times$ tail, **3.44× in the SR**)
- Processor injection, 3 bugs found and fixed, canary-validated
- Full vjets SR re-run (989 files) + merge
- Wired into **both** combine builders + method nuisance
- **Limit: 1742 $\rightarrow$ 1343 (v11), 1935 $\rightarrow$ 1491 (v32)**
- Provenance re-verified: live parquets reproduce from the Jul-15 model to 4×10^{-16}

Open / known

- **WtoLNu SR closure 1.133** — the largest single residual; worth a feature-coverage check of the W corner.
- $\sim 7\%$ of vjets SR events (689/10325) lost to transient xrootd errors on one site; re-run with it blacklisted before quoting a final number.
- **PCA over the per-bin ensemble covariance** (paper §IV D) instead of the event-level $\pm \delta g$ band — needs per-model P_+ re-dumped. Given the nuisance is already negligible ($\pm 1.3\%$), low priority.
- Training headroom (early stopping never fired).

What this did *not* fix

After negrw, v32 was re-optimised in four ways. **All four made the limit worse or did nothing** — recorded so they are not retried:

attempt	result
prune empty / low- N_{eff} bins	1491 → 1512
rebin / "log-transform" the discriminant	no config beats current (bracketed both sides)
blame the CRs for the MC-stat tax	refuted — it is 87% in the SR
free <code>tt</code> rateParam	1491 → 1542 (1581 with tt theory dropped)

Two lessons worth keeping:

1. `autoMCStats N` already applies `event-threshold=N` and **skips sub-threshold bins** — empty bins cost nothing, so pruning them buys nothing.
2. **Freezing a nuisance $\neq$ constraining it better.** A freeze scan measures how much freedom the fit *uses*; it is **not** a recoverable budget. Every "fix" derived from reading it that way failed.

The remaining v32 deficit is structural: it concentrates signal into bins that are ~72% tt. The one open lever is **more MC statistics at high D** — which is exactly what negrw bought.

Summary

The limit was MC-statistics-limited, not systematics-limited.

Learning $P_+(\vec{x})$ and estimating with $\sum |w|g$ instead of $\sum w$
raises N_{eff} by **3.44× in the SR** —

r_{95} : 1742 → 1343 (v11), 1935 → 1491 (v32), both -23%

Same yield, same shape, validated out-of-training-region,
with the method's own uncertainty included and shown to be negligible.

arXiv:2510.16217 · notes: [Projects/HToWW/negrw-training/](#)

Backup

Configuration reference

item	value
Training workflow	hww_genrw_train , 2022postEE
Training selection	base cuts + veto_emu_sr
Events / files	9,832,308 / 502 parquet
Positive fraction	0.836
Model	20 × HistGradientBoostingClassifier
Subsample	60%, without replacement, seeded
Hyperparameters	log_loss , max_iter 200, lr 0.05, max_depth 4, l2 1.0
Ensemble log-loss / AUC	0.331 / 0.829
Closure ratio (train region)	0.994
N_{eff} (train region)	2.92M → 4.68M (+60%)
N_{eff} (SR)	455 → 1563 (3.44×)
Image	coffea-base-almalinux9:0.7.30-py3.10 (sklearn 1.7.2)
Model artifact (master)	/eos/user/c/cgupta/HToWW/b-hive/negrw_out_img/negrw_models.joblib

Where it can fail

The identity $\sum |w|g \equiv \sum w$ holds for the **true** P_+ . With an estimated $\hat{P}_+$ the failure modes are:

- 1. Mis-modelled P_+** (poor features, under-trained). Symptom: closure ratio drifts from 1.
→ Gated by the closure test (0.994) and the SR calibration plot.
- 2. Extrapolation.** If the inference region sits outside the training domain in some feature, $\hat{P}_+$ is unconstrained there. → Gated by the explicit coverage check (interpolation only).
- 3. Wrong events reweighted.** A generator-specific g applied to a different process is silently wrong. → This is bug #1 above; gated by exact dataset matching.
- 4. Local closure $\neq$ global closure.** Even a well-calibrated $\hat{P}_+$ can drift in a small, kinematically-biased corner — this is our 6% SR offset, handled by the per-dataset renorm and declared as an extension beyond the paper.

Per-bin SR template, numerically

V+jets, SR_hplusc , from the combine input ROOT files.

bin	baseline $\sum w$	rel. err	negrw $\sum w g$	rel. err
1	21.5 ± 36.8	171%	63.9 ± 22.2	35%
2	151.4 ± 45.6	30%	70.9 ± 20.0	28%
3	153.3 ± 54.7	36%	139.3 ± 30.0	22%
4	221.3 ± 70.6	32%	191.1 ± 32.8	17%
5	205.6 ± 62.8	31%	172.1 ± 33.2	19%
6	0.00 ± 41.0	∞	83.4 ± 20.4	24%
7	35.8 ± 20.9	58%	25.3 ± 15.5	61%

Bin 6 is the bin that drove $r_{95} = 1742$: content floored to 10^{-6} by the builder's positivity guard, error ± 41 — an unconstrained background parameter directly under the signal.